

Planejamento técnico — Backend para integração autorizada com o LinkedIn

Versão: 1.0 — planejamento inicial

Status: documento de arquitetura; nenhum código implementado

Tecnologias obrigatórias: Java, Spring Boot, Spring Web, Spring Security quando necessário, Maven ou Gradle, SQLite, JPA/Hibernate, JDBC subjacente, APIs RESTful, OAuth 2.0 e Docker quando aplicável.

1. Resumo executivo

A primeira versão deve ser um backend próprio, modular e orientado a uma integração autorizada com o LinkedIn. O sistema não deve automatizar a navegação no site, coletar dados por scraping, usar cookies de sessão nem imitar cliques do usuário. Essas práticas são proibidas ou arriscadas segundo o Contrato do Usuário e a documentação de software proibido do LinkedIn ¹ ².

A integração inicial deve priorizar o produto **Share on LinkedIn**. A documentação oficial informa que a permissão aberta `w_member_social` permite criar publicações em nome do membro autenticado e que o compartilhamento utiliza a API UGC, com `POST https://api.linkedin.com/v2/ugcPosts` ³ ⁴. A aplicação deve tratar a publicação como uma operação explicitamente autorizada pelo usuário, com registro local e idempotência.

Busca, salvamento e candidatura a vagas não devem ser incluídos como operações automatizadas dentro do LinkedIn na primeira versão. A documentação pública consultada não apresenta uma permissão aberta para essas operações. Além disso, o LinkedIn aplica limites ao Easy Apply para conter bots e candidaturas rápidas ou inautênticas ⁵. O sistema pode, entretanto, manter uma lista própria de oportunidades obtidas de fontes autorizadas e permitir que o usuário revise e abra os links manualmente.

A arquitetura recomendada é uma **arquitetura hexagonal leve**, também chamada de ports and adapters. Ela separa o domínio e as regras de negócio dos detalhes do LinkedIn, do Spring MVC e do SQLite. Isso facilita testes, mudanças na API externa e uma futura migração para PostgreSQL.

2. Escopo e limites da primeira versão

2.1 Escopo funcional recomendado

A versão inicial deve oferecer:

1. cadastro lógico do usuário local da aplicação;

2. início e conclusão do fluxo OAuth 2.0 com o LinkedIn;
3. armazenamento seguro da autorização do LinkedIn;
4. obtenção dos dados mínimos de identidade necessários para associar a autorização ao usuário local;
5. criação de rascunhos de publicação;
6. revisão e aprovação explícita de rascunhos;
7. publicação de texto, URL e, em uma etapa posterior, mídia conforme os recursos documentados pelo produto Share on LinkedIn;
8. histórico das publicações iniciadas pela aplicação;
9. tratamento de erros, expiração de tokens, limites e falhas transitórias;
10. API REST própria, documentada com OpenAPI;
11. testes automatizados e documentação adequada para portfólio.

2.2 Fora do escopo inicial

Não devem ser implementados, sem documentação e aprovação específicas do LinkedIn:

- scraping de perfis, vagas, publicações ou resultados de pesquisa;
- login automatizado por navegador, uso de cookies ou automação de DOM;
- envio automático de convites ou mensagens;
- automação de curtidas, comentários ou compartilhamentos fora dos endpoints e permissões confirmados;
- busca de vagas diretamente pela interface do LinkedIn;
- salvamento automático de vagas dentro da conta do LinkedIn;
- candidatura automática, inclusive Easy Apply;
- automação de Sales Navigator;
- coleta ampla de analytics, conexões ou dados de terceiros;
- publicação sem aprovação ou regra de autorização local claramente definida.

3. Validação da API oficial do LinkedIn

3.1 Capacidade confirmada para a primeira versão

Funcionalidade	Produto ou endpoint	Permissão	Aprovação especial	Limitação arquitetural
----------------	---------------------	-----------	--------------------	------------------------

Autenticar o membro	OAuth 2.0 do LinkedIn	Escopos solicitados pelo aplicativo	O aplicativo precisa ser registrado e autorizado pelo membro	O backend deve validar <code>state</code> , redirecionamento e troca segura do código
Criar publicação	Share on LinkedIn; <code>POST /v2/ugcPosts</code>	<code>w_member_social</code>	A documentação o classifica como permissão aberta; ainda é necessário configurar o produto no Developer Portal	A publicação é imediata quando <code>lifecycleState</code> é <code>PUBLISHED</code> ; o agendamento deve ser responsabilidade da aplicação, não presumido como recurso da API
Publicar texto	Share on LinkedIn	<code>w_member_social</code>	Não indicada aprovação de parceiro na documentação consultada	O corpo deve usar o formato oficial e a visibilidade aceita pela API
Compartilhar URL	Share on LinkedIn	<code>w_member_social</code>	Não indicada aprovação de parceiro na documentação consultada	A URL e os metadados devem ser enviados no formato documentado
Publicar imagem ou vídeo	Share on LinkedIn	<code>w_member_social</code>	Confirmar no momento da implementação os requisitos atuais de upload e o produto habilitado	É necessário registrar e enviar a mídia antes de criar a publicação

A documentação geral de acesso informa que as permissões abertas disponíveis a todos os desenvolvedores incluem **Share on LinkedIn** com `w_member_social` , descrita como permissão para publicar, comentar e curtir em nome do membro autenticado ⁴ . Isso não deve ser interpretado como autorização para automatizar ações por navegador ou como autorização para qualquer operação não documentada. Cada ação deve ser implementada somente quando o endpoint, o corpo, a permissão e os limites estiverem confirmados na documentação atual.

3.2 Capacidades que exigem confirmação ou aprovação

Marketing, Sales Navigator, Talent, Compliance e outros produtos possuem permissões e programas próprios. A documentação de acesso informa que muitas permissões exigem aprovação explícita ou participação em programas de parceiros ⁴. O projeto deve manter uma matriz de capacidades versionada, com as seguintes colunas: produto, endpoint, escopo, aprovação, finalidade, retenção autorizada, limite e data da última verificação.

Não se deve implementar uma funcionalidade apenas porque ela existe no site do LinkedIn. O fato de um usuário conseguir pesquisar, salvar ou se candidatar manualmente não demonstra a existência de uma API pública para a mesma operação.

3.3 Vagas de emprego

A versão inicial deve tratar vagas como uma fonte externa ou como uma entrada manual do usuário. O backend pode armazenar uma oportunidade que o usuário tenha inserido ou que venha de uma fonte autorizada, desde que os termos dessa fonte permitam tal uso. O sistema pode calcular compatibilidade com currículo, classificar oportunidades e guardar o status local.

O sistema não deve abrir uma sessão automatizada do LinkedIn para pesquisar, salvar ou candidatar-se. A política do LinkedIn proíbe bots, crawlers, extensões e outros meios automatizados que raspem ou automatizem atividades no site ¹ ². O LinkedIn também informa que candidaturas rápidas podem acionar pausa temporária e que os limites do Easy Apply existem, entre outros motivos, para combater automação e bots ⁵.

O fluxo permitido e recomendado é: coletar de fonte autorizada, filtrar localmente, apresentar ao usuário, abrir o link por ação do usuário e deixar o salvamento ou a candidatura sob controle manual.

4. Arquitetura proposta

4.1 Princípios

O sistema deve aplicar separação de responsabilidades, baixo acoplamento, alta coesão, SOLID, Clean Code e DDD apenas onde ele realmente reduzir a complexidade. O domínio não deve conhecer classes de resposta do LinkedIn, entidades JPA ou detalhes de HTTP.

A aplicação deve ser organizada em quatro áreas:

1. **Domínio:** regras, entidades de negócio, objetos de valor e portas.
2. **Aplicação:** casos de uso, transações e orquestração.
3. **Adaptadores de entrada:** controllers REST, autenticação e validação de requests.

4. **Adaptadores de saída:** JPA, cliente HTTP do LinkedIn, armazenamento de segredos e observabilidade.

4.2 Estrutura de pacotes sugerida

Plain Text

```
com.example.linkedinagent
├── domain
│   ├── user
│   ├── linkedin
│   ├── publishing
│   ├── opportunity
│   └── common
├── application
│   ├── auth
│   ├── publishing
│   ├── opportunity
│   └── ports
│       ├── in
│       └── out
├── adapter
│   ├── in
│   │   └── web
│   └── out
│       ├── linkedin
│       ├── persistence
│       └── security
├── config
├── exception
└── shared
```

Uma alternativa mais simples, adequada ao primeiro commit, é organizar por feature, mantendo dentro de cada feature os pacotes `domain`, `application` e `adapter`. O ponto essencial é que os DTOs externos do LinkedIn fiquem dentro do adaptador `linkedin` e nunca atravessem a fronteira do caso de uso.

4.3 Portas e adaptadores

Portas de entrada podem incluir:

- `StartLinkedInAuthorizationUseCase` ;
- `CompleteLinkedInAuthorizationUseCase` ;
- `CreateDraftUseCase` ;
- `ApproveDraftUseCase` ;

- `PublishApprovedDraftUseCase` ;
- `ListPublicationHistoryUseCase` ;
- `RegisterOpportunityUseCase` .

Portas de saída podem incluir:

- `LinkedInAuthorizationPort` ;
- `LinkedInProfilePort` ;
- `LinkedInPublishingPort` ;
- `AccessTokenRepository` ;
- `UserRepository` ;
- `DraftRepository` ;
- `PublicationRepository` ;
- `ClockPort` para testes determinísticos.

A implementação HTTP do LinkedIn deve converter respostas externas em modelos internos. O domínio deve trabalhar, por exemplo, com `LinkedInPublicationCommand` e `PublicationResult` , e não com `com.linkedin.ugc.ShareContent` ou mapas JSON.

5. Fluxo OAuth 2.0

5.1 Fluxo recomendado

1. O usuário autenticado na aplicação solicita a conexão com o LinkedIn.
2. O backend gera um `state` criptograficamente aleatório, curto e de uso único.
3. O backend redireciona o navegador para a autorização do LinkedIn com `client_id` , `redirect_uri` , `response_type=code` , `state` e os escopos necessários.
4. O membro autoriza o aplicativo no LinkedIn.
5. O LinkedIn redireciona para o callback do backend com `code` e `state` .
6. O backend compara o `state` com o valor armazenado e rejeita reutilização ou divergência.
7. O backend troca o código por um access token usando comunicação TLS e credenciais mantidas em segredo.
8. O backend consulta apenas os dados mínimos necessários para identificar o membro e associa a autorização ao usuário local.

9. O backend armazena o token de forma protegida e remove o `code` e o `state` temporários.

10. A aplicação informa ao usuário que a conexão foi concluída.

O frontend nunca deve receber o `client_secret`, o refresh token ou um access token de longa duração. O callback deve ser processado pelo backend.

5.2 Tokens e sessões

O modelo deve armazenar o token apenas se a aplicação precisar realizar chamadas posteriores. Devem ser armazenados o mínimo necessário: referência do usuário local, identificador do membro quando permitido, access token cifrado, tipo, escopos concedidos, data de emissão, data de expiração quando fornecida, data da última utilização e status de revogação.

Refresh tokens não devem ser presumidos. A aplicação deve verificar na documentação e na resposta efetiva do fluxo se o produto concede refresh token. Se existir, ele deve receber o mesmo tratamento de um segredo de alto valor. Se não existir, o sistema deve marcar a autorização como expirada e exigir nova autorização.

Não devem ser armazenados: senha do LinkedIn, cookies, dados completos de sessão do navegador, código OAuth depois da troca, tokens em logs, respostas externas completas sem finalidade definida ou dados de terceiros além do necessário.

5.3 Proteções de segurança

A configuração deve incluir:

- `state` de uso único contra CSRF no OAuth;
- `redirect_uri` exata, sem curingas;
- HTTPS em produção;
- segredo do cliente em variável de ambiente ou secret manager;
- cifragem em repouso dos tokens, com chave fora do banco;
- rotação planejada de chaves;
- logs com mascaramento de tokens e identificadores;
- expiração de sessão local;
- autorização por usuário e, futuramente, por papéis;
- CORS restrito às origens conhecidas;
- CSRF habilitado para sessões baseadas em cookie, ou justificativa documentada se a API usar somente bearer tokens sem sessão de navegador;

- limites de requisição nos endpoints de OAuth e publicação;
- validação de tamanho, tipo e conteúdo dos campos;
- tratamento uniforme de falhas sem revelar detalhes internos.

6. Modelo de dados SQLite com JPA/Hibernate

6.1 Entidades principais

Entidade	Finalidade	Atributos essenciais
AppUser	Usuário da aplicação	id , email , displayName , status , createdAt , updatedAt
LinkedInAuthorization	Autorização concedida ao usuário	id , appId , memberSubject , encryptedAccessToken , encryptedRefreshToken opcional, scopes , expiresAt , status , createdAt , updatedAt
ContentDraft	Rascunho editável	id , appId , text , mediaCategory , originalUrl , title , status , approvedAt , createdAt , updatedAt
Publication	Histórico de uma tentativa ou publicação	id , appId , draftId , externalPostId , requestFingerprint , status , failureCode , publishedAt , createdAt
Opportunity	Vaga ou oportunidade mantida localmente	id , appId , source , externalUrl , title , companyName , location , descriptionSnapshot opcional, matchScore opcional, status , foundAt , expiresAt opcional
OAuthTransaction	Estado temporário do OAuth	id , appId , stateHash , scopes , expiresAt , consumedAt , createdAt
AuditEvent	Auditoria de ações relevantes	id , appId , eventType , resourceType , resourceId , outcome , occurredAt , metadataSanitized

`LinkedInAuthorization` deve ter uma restrição única para impedir duas autorizações ativas do mesmo usuário e, quando aplicável, uma restrição para o identificador externo do membro. Tokens devem ser cifrados antes de persistir. O segredo de cifragem não pode ficar na mesma tabela nem no repositório Git.

`ContentDraft` deve guardar somente o conteúdo que o usuário deseja gerenciar. Se a aplicação armazenar uma descrição de vaga, deve aplicar minimização, retenção e base legal adequadas. O snapshot de uma página externa não deve ser tratado como necessário por padrão.

6.2 Relacionamentos JPA

- `AppUser` possui `@OneToMany(mappedBy = "appUser")` com autorizações, rascunhos, publicações e oportunidades.
- `LinkedInAuthorization` possui `@ManyToOne(fetch = LAZY)` com `AppUser`.
- `ContentDraft` possui `@ManyToOne(fetch = LAZY)` com `AppUser`.
- `Publication` possui `@ManyToOne(fetch = LAZY)` com `AppUser` e, opcionalmente, com `ContentDraft`.
- `OAuthTransaction` possui `@ManyToOne(fetch = LAZY)` com `AppUser`.
- `AuditEvent` possui `@ManyToOne(fetch = LAZY)` com `AppUser`.

Não usar `EAGER` por padrão. Preferir consultas específicas, projeções e `JOIN FETCH` controlado quando necessário. Evitar `@ManyToMany` na primeira versão; escopos podem ser armazenados como coleção simples validada ou como uma tabela própria se houver necessidade de consulta por escopo.

6.3 SQLite e migração

SQLite é adequado para desenvolvimento e para uma instalação pequena, mas possui diferenças importantes em relação a bancos servidor. A aplicação deve:

- usar tipos portáteis e tamanhos de coluna razoáveis;
- evitar SQL específico de SQLite na camada de domínio;
- evitar dependência em `rowid`, `WITHOUT ROWID`, triggers proprietárias e funções específicas;
- usar IDs gerados pela aplicação ou estratégia compatível;
- configurar transações curtas;
- habilitar foreign keys na conexão;

- considerar o comportamento de concorrência e o bloqueio de escrita;
- evitar guardar objetos JSON como única representação de dados que precisam ser consultados;
- usar `TEXT` para timestamps ISO-8601 ou uma estratégia Hibernate testada e documentada;
- manter índices explícitos para chaves de busca;
- usar migrations versionadas, preferencialmente Flyway, validando cada migration contra SQLite.

Hibernate pode funcionar com SQLite por meio de um dialeto compatível, mas o dialeto, a geração de schema e operações de alteração de tabela devem ser testados. `ddl-auto=update` não deve ser utilizado em produção. Em desenvolvimento inicial, `validate` com migrations versionadas é mais previsível.

7. API REST própria

A API deve ser versionada desde o começo com o prefixo `/api/v1`.

7.1 Endpoints iniciais

Método e rota	Finalidade	Autenticação	Resposta principal
<code>GET</code> <code>/api/v1/linkedin/oauth/start</code>	Iniciar OAuth	Usuário local	<code>302 Found</code> para o LinkedIn
<code>GET</code> <code>/api/v1/linkedin/oauth/callback</code>	Processar callback	<code>state</code> válido	<code>302 Found</code> ou resposta de conclusão
<code>GET</code> <code>/api/v1/linkedin/connection</code>	Ver estado da conexão	Usuário local	<code>200</code> com estado sanitizado
<code>DELETE</code> <code>/api/v1/linkedin/connection</code>	Revogar conexão local	Usuário local	<code>204</code>
<code>POST</code> <code>/api/v1/drafts</code>	Criar rascunho	Usuário local	<code>201</code>
<code>GET</code> <code>/api/v1/drafts</code>	Listar rascunhos paginados	Usuário local	<code>200</code>
<code>GET</code> <code>/api/v1/drafts/{id}</code>	Consultar rascunho	Usuário local	<code>200</code>

PATCH /api/v1/drafts/{id}	Editar rascunho	Usuário local	200
POST /api/v1/drafts/{id}/approve	Aprovar publicação	Usuário local	200
POST /api/v1/drafts/{id}/publish	Publicar rascunho aprovado	Usuário local	202 ou 201
GET /api/v1/publications	Histórico de publicações	Usuário local	200
POST /api/v1/opportunities	Registrar oportunidade local	Usuário local	201
GET /api/v1/opportunities	Filtrar oportunidades próprias	Usuário local	200
PATCH /api/v1/opportunities/{id}	Atualizar status local	Usuário local	200
GET /actuator/health	Health check	Conforme exposição configurada	200 ou 503

O endpoint de publicação deve verificar propriedade do rascunho, status `APPROVED`, existência de autorização ativa e ausência de publicação anterior com a mesma impressão digital de requisição. Deve exigir um cabeçalho de idempotência, como `Idempotency-Key`, para evitar duplicação em retries.

7.2 DTOs e erros

Os DTOs de entrada devem usar Bean Validation. O domínio não deve receber `@RequestBody` diretamente. A resposta de erro deve seguir um formato consistente, como `application/problem+json`, contendo `type`, `title`, `status`, `detail`, `instance`, `traceId` e uma lista de erros de campo quando aplicável.

Exemplos de códigos:

- 400 para payload inválido;
- 401 para ausência de autenticação;
- 403 para falta de autorização;
- 404 para recurso inexistente do usuário atual;
- 409 para conflito de estado ou idempotência;
- 422 para rascunho não publicável;

- 429 para limite local ou externo;
- 502 ou 503 para falha transitória do LinkedIn;
- 500 apenas para falhas inesperadas, sem vaziar stack trace.

8. Regras de negócio

Os estados mínimos de `ContentDraft` devem ser `DRAFT`, `APPROVED`, `PUBLISHING`, `PUBLISHED` e `FAILED`. Transições inválidas devem ser rejeitadas pelo domínio.

A publicação deve ser uma operação explícita. A aplicação pode permitir agendamento local, mas o worker ou executor só poderá enviar conteúdo que esteja aprovado, dentro do horário definido e com autorização válida. A aplicação não deve afirmar que o LinkedIn possui agendamento nativo se isso não estiver documentado para o produto e permissão usados.

A regra de idempotência deve combinar usuário, rascunho, versão do conteúdo e uma chave fornecida pelo cliente. Se uma resposta externa for perdida depois da publicação, uma repetição não deve criar outra publicação automaticamente sem uma estratégia de reconciliação segura.

9. Testes

9.1 Unitários

JUnit e Mockito devem cobrir:

- transições de estado de rascunho;
- validação de autorização;
- decisão de publicação;
- cálculo de idempotência;
- tratamento de token expirado;
- conversores entre modelos internos e DTOs do LinkedIn;
- mapeadores de erro;
- regras de retenção de oportunidades.

9.2 Integração

Testes de integração devem usar Spring Boot Test, MockMvc e uma base SQLite real em arquivo temporário ou memória conforme a compatibilidade do driver. O objetivo é validar mappings, constraints, migrations, transações e consultas.

A integração HTTP com o LinkedIn deve usar WireMock ou MockWebServer. Os cenários devem incluir sucesso, 401, 403, 409, 429, timeout, JSON inesperado, token expirado e erro de upload de mídia.

O projeto deve testar o callback OAuth com state correto, incorreto, expirado e reutilizado. Também deve confirmar que nenhum token aparece nos logs e que um usuário não consegue acessar o rascunho de outro usuário.

9.3 Testes de controller e segurança

MockMvc deve verificar autenticação, autorização, validação, códigos HTTP, headers de correlação, resposta de erro e CORS. Controllers devem ser finos; a lógica deve ser testada nos casos de uso.

10. Observabilidade e qualidade

A aplicação deve usar logs estruturados com `traceld` e `requestId`. Logs de publicação devem registrar usuário local, rascunho, operação, resultado e latência, mas nunca tokens, códigos OAuth, conteúdo sensível desnecessário ou respostas completas do LinkedIn.

Spring Boot Actuator deve expor somente health checks necessários. Métricas úteis incluem latência por operação, sucesso e falha de publicação, respostas 429, tokens expirados e tamanho da fila local. Endpoints de observabilidade não devem ser públicos sem proteção.

A documentação deve usar springdoc-openapi. O README deve incluir diagrama, fluxo OAuth, instruções de configuração, limitações conhecidas, modelo de dados, exemplos curl sanitizados, como executar testes e como revogar a conexão.

Docker deve ser utilizado para padronizar o ambiente de desenvolvimento, com volume persistente para SQLite e secrets fornecidos por ambiente. A imagem não deve conter `.env`, tokens ou banco com dados reais.

11. LGPD e privacidade

Este documento não substitui orientação jurídica. Tecnicamente, a aplicação deve definir finalidade, necessidade, retenção e controle de acesso antes de armazenar dados do LinkedIn.

A coleta deve ser mínima e limitada ao funcionamento declarado. O sistema deve separar dados do usuário local, tokens de autorização, dados de publicação e oportunidades. Dados de terceiros, como informações de recrutadores ou outros membros, não devem ser coletados por scraping.

O usuário deve conseguir desconectar o LinkedIn. A desconexão deve apagar ou inutilizar tokens, revogar a autorização quando houver mecanismo suportado e registrar o evento sem guardar o segredo. Deve existir uma rotina de exclusão ou anonimização de dados locais conforme a finalidade e a política de retenção.

O banco deve ter controle de acesso por usuário. Backups devem ser cifrados. O sistema deve documentar quais dados vêm do LinkedIn, quais são criados pelo usuário e quais são derivados pela aplicação. O acesso interno aos tokens deve ficar restrito ao adaptador responsável pela integração.

12. Migração futura para PostgreSQL

A migração deve ser tratada como mudança de adaptador de persistência e configuração, não como alteração do domínio. Para isso:

1. usar JPA e interfaces de repositório, sem SQL específico nas regras de negócio;
2. evitar tipos e funções exclusivas do SQLite;
3. manter migrations compatíveis ou separar scripts por banco quando necessário;
4. usar nomes de tabela e coluna explícitos e portáteis;
5. evitar dependência em comportamento de `NULL`, ordenação implícita ou chaves geradas específicas;
6. testar constraints e transações em ambos os bancos;
7. externalizar URL, usuário, senha e driver por perfil;
8. substituir o driver SQLite pelo PostgreSQL no perfil de produção;
9. trocar o dialeto Hibernate e revisar migrations;
10. revisar concorrência, pooling e isolamento de transações.

As entidades, serviços, controllers, portas e regras de negócio não deveriam precisar ser reescritos. Os pontos mais prováveis de ajuste são dependência JDBC, propriedades de datasource, dialeto Hibernate, migrations e comportamento de concorrência.

13. Configuração por ambiente

Devem existir perfis `local`, `test` e `prod`. Valores sensíveis devem vir de variáveis de ambiente:

Plain Text

```
LINKEDIN_CLIENT_ID
LINKEDIN_CLIENT_SECRET
```

```
LINKEDIN_REDIRECT_URI
TOKEN_ENCRYPTION_KEY
APP_BASE_URL
DATABASE_URL
DATABASE_USERNAME
DATABASE_PASSWORD
CORS_ALLOWED_ORIGINS
```

O `application.yml` pode conter defaults não sensíveis para o perfil local, mas não deve conter o segredo real. O perfil de teste deve usar credenciais fictícias e servidor HTTP simulado.

14. Roadmap recomendado

Etapa 1 — validação documental

Objetivo: confirmar os produtos, endpoints, permissões, limites e políticas atuais.

Resultado: matriz de capacidades com fonte, data de verificação e escopo.

Critério de conclusão: nenhuma funcionalidade planejada depende de endpoint inventado.

Etapa 2 — aplicação LinkedIn

Objetivo: criar o aplicativo no Developer Portal e habilitar somente Share on LinkedIn e os escopos indispensáveis.

Resultado: client ID, redirect URI e ambiente de desenvolvimento configurados.

Critério: OAuth manual funciona em ambiente local sem expor o segredo.

Etapa 3 — arquitetura e bootstrap

Objetivo: criar o projeto Spring Boot com Maven ou Gradle, Java LTS, profiles, dependências e estrutura de pacotes.

Resultado: aplicação inicia, possui health check e pipeline local de testes.

Critério: build limpo e README inicial.

Etapa 4 — persistência SQLite

Objetivo: configurar driver JDBC, JPA/Hibernate, migrations e entidades iniciais.

Resultado: schema versionado e repositories testados.

Critério: testes de persistência passam em SQLite real.

Etapa 5 — identidade e OAuth

Objetivo: implementar usuário local, state de uso único, callback, token cifrado e desconexão.

Resultado: autorização completa sem token no frontend ou logs.

Critério: testes de sucesso, expiração, replay e erro passam.

Etapa 6 — adaptador LinkedIn

Objetivo: implementar cliente HTTP isolado e mapeadores para o Share API.

Resultado: publicação de texto e URL em ambiente autorizado.

Critério: testes simulados cobrem respostas de sucesso e falha.

Etapa 7 — rascunhos e publicação

Objetivo: criar estados, aprovação, idempotência, histórico e retry seguro.

Resultado: usuário cria, aprova e publica um rascunho.

Critério: não há publicação duplicada em retry conhecido.

Etapa 8 — oportunidades locais

Objetivo: permitir registrar e classificar vagas vindas de fontes autorizadas ou entrada manual.

Resultado: lista própria de oportunidades sem scraping do LinkedIn.

Critério: o sistema não precisa acessar a sessão do LinkedIn para funcionar.

Etapa 9 — API REST e documentação

Objetivo: concluir controllers, DTOs, validação, erros, OpenAPI e versionamento.

Resultado: contrato REST documentado e testado.

Critério: exemplos do README executam em ambiente local.

Etapa 10 — segurança, privacidade e observabilidade

Objetivo: revisar CORS, CSRF conforme o modelo de sessão, rate limits, logs, auditoria, exclusão e retenção.

Resultado: checklist de segurança e LGPD técnica.

Critério: revisão não encontra tokens em logs nem acesso cruzado entre usuários.

Etapa 11 — Docker e qualidade de portfólio

Objetivo: padronizar execução, adicionar diagrama, documentação, testes e CI futuro.

Resultado: repositório apresentável e reproduzível.

Critério: clone novo consegue iniciar o projeto seguindo o README, sem credenciais reais versionadas.

Etapa 12 — preparação para PostgreSQL

Objetivo: validar que o domínio permanece independente do banco.

Resultado: perfil experimental PostgreSQL e relatório das diferenças.

Critério: migrations e testes principais passam nos dois bancos antes de qualquer migração de produção.

15. Ordem de implementação final

A ordem recomendada é: validar a documentação; criar e configurar o aplicativo LinkedIn; definir o contrato de segurança; inicializar o Spring Boot; configurar SQLite e migrations; modelar usuários e autorizações; implementar OAuth; criar a porta e o adaptador de publicação; criar rascunhos e histórico; expor a API REST; adicionar oportunidades locais; implementar testes; adicionar observabilidade; documentar; empacotar com Docker; e somente depois avaliar PostgreSQL ou novos produtos do LinkedIn.

Essa ordem reduz retrabalho porque valida primeiro a capacidade real da plataforma, estabiliza a fronteira externa antes das regras de negócio e mantém a persistência substituível. Ela também produz um projeto de portfólio demonstrável mesmo que o LinkedIn altere ou limite futuras permissões.

16. Roteiro operacional cronológico

Esta seção transforma a arquitetura em uma sequência executável. Cada passo deve ser concluído antes do próximo, salvo quando o documento indicar uma atividade paralela. O agente de IA responsável por uma etapa deve ler o documento inteiro antes de alterar o repositório, respeitar as restrições da seção 2 e não inventar capacidades da API do LinkedIn.

16.1 Regras para qualquer agente de IA

Antes de modificar o projeto, o agente deve:

1. confirmar o objetivo da etapa atual;
2. inspecionar a estrutura existente, o branch ativo, o build e os testes;
3. ler o `README`, as configurações e as migrations já existentes;
4. procurar implementações duplicadas antes de criar novos arquivos;
5. propor alterações pequenas e coerentes com a arquitetura;
6. nunca adicionar tokens, senhas, cookies, arquivos `.env` reais ou dados pessoais ao Git;
7. não implementar endpoint, scope ou produto do LinkedIn que não esteja confirmado na documentação oficial;

8. escrever ou atualizar testes junto com a alteração;
9. executar o build e os testes relevantes;
10. registrar no relatório final os arquivos alterados, comandos executados, resultado dos testes, riscos e o próximo passo.

O agente não deve avançar automaticamente quando um teste falhar, uma migration for destrutiva, uma permissão do LinkedIn estiver indefinida ou uma decisão puder alterar o modelo de segurança. Nesses casos, deve registrar a dúvida e solicitar decisão antes de continuar.

16.2 Passo 0 — criar o repositório e o contrato de trabalho

Objetivo: estabelecer um repositório reproduzível e regras para futuras contribuições humanas e automatizadas.

Ações: criar o repositório privado ou público conforme a decisão do proprietário; adicionar README.md , LICENSE quando aplicável, .gitignore , CONTRIBUTING.md , SECURITY.md e este planejamento; definir o branch principal; registrar versão do Java, ferramenta de build e convenções de commit.

Entregáveis: repositório inicial, documentação de contribuição e checklist de segurança.

Verificação: um clone limpo contém somente arquivos não sensíveis e explica como executar o projeto, mesmo que a aplicação ainda não exista.

Instrução para o agente: não gerar código de domínio nesta etapa. Primeiro crie a estrutura documental e confirme as decisões com o estado real do repositório.

16.3 Passo 1 — validar a API e congelar o escopo da versão 1

Objetivo: transformar a pesquisa da API em decisões implementáveis.

Ações: revisar as fontes oficiais; registrar o produto Share on LinkedIn, o scope w_member_social , o endpoint de publicação, o fluxo OAuth e os limites conhecidos; separar funcionalidades confirmadas, dependentes de aprovação, desconhecidas e proibidas; decidir que vagas serão tratadas somente como oportunidades locais ou vindas de fontes autorizadas.

Entregáveis: docs/linkedin-capability-matrix.md , lista de escopos permitidos e decisão formal de escopo.

Verificação: cada chamada planejada tem fonte oficial, método HTTP, endpoint, payload, permissão, tratamento de erro e regra de armazenamento. Capacidades sem confirmação não entram no backlog de implementação.

Instrução para o agente: se a documentação encontrada contradizer este plano, não escolher silenciosamente. Atualizar a matriz, destacar a divergência e solicitar revisão da

decisão.

16.4 Passo 2 — registrar o aplicativo no LinkedIn sem expor segredos

Objetivo: preparar credenciais de desenvolvimento para OAuth.

Ações: criar o aplicativo no Developer Portal; habilitar somente os produtos necessários; configurar uma redirect URI local exata; guardar client ID e client secret fora do Git; documentar a configuração em `.env.example` sem valores reais.

Entregáveis: aplicação registrada, variáveis de ambiente documentadas e redirect URI definida.

Verificação: o client secret não aparece no histórico Git, nos logs ou na documentação pública. A aplicação local consegue iniciar sem credenciais reais apenas nos testes.

Dependência: conclusão do Passo 1.

16.5 Passo 3 — gerar o esqueleto Spring Boot

Objetivo: criar uma base compilável e pequena.

Ações: configurar Java LTS, Spring Boot, Spring Web, Spring Security quando o modelo de autenticação estiver definido, validação, Actuator, JPA, driver JDBC do SQLite, ferramenta de migrations, cliente HTTP e springdoc OpenAPI. Criar os pacotes previstos sem criar entidades vazias sem uso.

Entregáveis: aplicação que inicia, endpoint de health check, build configurado e teste de contexto.

Verificação: `./mvnw test` ou `./gradlew test` passa em máquina limpa; o projeto inicia com configuração de teste sem LinkedIn real.

Instrução para o agente: não usar `ddl-auto=update` como solução de versionamento e não colocar a lógica da integração dentro de controllers.

16.6 Passo 4 — configurar ambientes e segredos

Objetivo: separar local, teste e produção desde o primeiro commit funcional.

Ações: criar profiles; definir propriedades não sensíveis; documentar `LINKEDIN_CLIENT_ID`, `LINKEDIN_CLIENT_SECRET`, `LINKEDIN_REDIRECT_URI` e `TOKEN_ENCRYPTION_KEY`; configurar logs mascarados; definir origens CORS esperadas; criar configuração de teste com valores fictícios.

Entregáveis: `application.yml` ou equivalente por profile, `.env.example` e documentação de configuração.

Verificação: o profile de teste funciona sem segredo real; propriedades obrigatórias falham de modo claro quando ausentes; nenhum segredo é impresso no startup.

16.7 Passo 5 — criar o schema SQLite e as migrations

Objetivo: estabelecer persistência reproduzível antes das regras de negócio.

Ações: configurar conexão JDBC, foreign keys, transações e migrations; criar primeiro as tabelas de usuário, autorização OAuth e transação OAuth; depois criar rascunho, publicação, oportunidade e auditoria; adicionar constraints e índices necessários.

Entregáveis: migrations versionadas e entidades JPA com mapeamentos explícitos.

Verificação: um banco vazio sobe todas as migrations; uma segunda execução não duplica schema; testes confirmam chaves estrangeiras, unicidade e carregamento LAZY.

Instrução para o agente: documentar toda decisão específica de SQLite e indicar se ela dificulta PostgreSQL. Não criar colunas para dados que ainda não têm finalidade.

16.8 Passo 6 — implementar identidade local e autorização da aplicação

Objetivo: garantir que cada recurso pertença a um usuário local antes de conectar ao LinkedIn.

Ações: escolher o mecanismo de autenticação local da API; implementar resolução do usuário atual; criar autorização por proprietário do recurso; proteger controllers; adicionar testes de acesso cruzado.

Entregáveis: `AppUser`, mecanismo de sessão ou bearer token local, filtro ou configuração de segurança e regras de autorização.

Verificação: usuário não autenticado recebe `401`; usuário autenticado não acessa dados de outro usuário; recursos inexistentes não revelam informações indevidas.

16.9 Passo 7 — implementar o ciclo OAuth com o LinkedIn

Objetivo: conectar e desconectar um membro usando OAuth 2.0 sem expor credenciais.

Ações: criar o caso de uso de início; gerar e persistir hash do `state` com expiração; implementar callback; trocar o authorization code; validar resposta; salvar tokens cifrados; consultar somente os dados mínimos necessários; implementar desconexão e expiração.

Entregáveis: `LinkedInAuthorization`, `OAuthTransaction`, porta de autorização e adaptador HTTP.

Verificação: testes cobrem callback válido, state incorreto, state reutilizado, state expirado, erro de troca, token expirado e ausência de refresh token. Logs não contêm code, token ou

secret.

Instrução para o agente: não enviar token ao frontend. Não presumir que refresh token existe; verificar a resposta e a documentação do produto.

16.10 Passo 8 — implementar o adaptador de publicação

Objetivo: encapsular toda a comunicação de publicação com o LinkedIn.

Ações: criar DTOs externos somente no pacote do adaptador; criar mapeadores; implementar publicação de texto; depois URL; deixar mídia para uma etapa separada; configurar timeout, headers, correlação, tratamento de status e conversão de erros.

Entregáveis: `LinkedInPublishingPort` , cliente HTTP, DTOs externos e testes com WireMock ou MockWebServer.

Verificação: o domínio não importa classes do LinkedIn; testes cobrem `201` , `400` , `401` , `403` , `409` , `429` , timeout e JSON inesperado.

16.11 Passo 9 — implementar rascunhos, aprovação e idempotência

Objetivo: impedir publicação acidental ou duplicada.

Ações: criar estados do rascunho; criar endpoints e casos de uso de criação e edição; exigir aprovação explícita; validar autorização ativa antes de publicar; adicionar `Idempotency-Key` ; gravar tentativas e resultado; implementar retry somente para falhas transitórias.

Entregáveis: fluxo completo de `DRAFT` até `PUBLISHED` ou `FAILED` .

Verificação: rascunho não aprovado não publica; rascunho de outro usuário não publica; retry da mesma chave não cria duplicata; falha permanente não entra em loop.

16.12 Passo 10 — adicionar oportunidades locais sem scraping

Objetivo: apoiar busca de vagas sem automatizar o LinkedIn.

Ações: criar endpoint para entrada manual e, somente depois de validar termos da fonte, adaptadores para feeds ou APIs autorizadas; armazenar URL, origem, título, empresa, localização e status local; implementar filtros e pontuação de compatibilidade; não salvar cookies nem conteúdo raspado do LinkedIn.

Entregáveis: entidade `Opportunity` , casos de uso, API REST e política de retenção.

Verificação: o recurso funciona sem login automatizado no LinkedIn; cada fonte possui autorização documentada; a candidatura continua sendo uma decisão manual do usuário.

16.13 Passo 11 — finalizar a API REST e o contrato OpenAPI

Objetivo: tornar o backend consumível por frontend ou outro agente.

Ações: finalizar DTOs, validações, paginação, ordenação, códigos HTTP, Problem Details , versionamento e documentação OpenAPI; adicionar exemplos seguros; padronizar headers de correlação e idempotência.

Entregáveis: especificação OpenAPI, controllers finos e README com exemplos de uso.

Verificação: todos os endpoints têm autenticação e autorização documentadas; erros de validação têm formato estável; exemplos não usam tokens reais.

16.14 Passo 12 — completar testes automatizados

Objetivo: provar o comportamento das camadas antes de adicionar complexidade operacional.

Ações: completar testes unitários de domínio e serviços; testes de repository com SQLite real; testes de controller com MockMvc; testes OAuth; testes do cliente HTTP; testes de autorização, rate limit e auditoria.

Entregáveis: suíte automatizada e relatório de cobertura usado como indicador, não como único critério de qualidade.

Verificação: build completo passa sem rede externa; testes são determinísticos; falhas são reproduzíveis; nenhum teste depende de uma conta real do LinkedIn.

16.15 Passo 13 — aplicar segurança, LGPD técnica e observabilidade

Objetivo: revisar o sistema como produto que manipula credenciais e dados pessoais.

Ações: executar checklist de segredo, cifragem, logs, CORS, CSRF, headers, rate limit, retenção, exclusão, auditoria e backup; configurar métricas e health checks; testar expiração e revogação.

Entregáveis: SECURITY.md , política técnica de retenção, matriz de dados e logs estruturados.

Verificação: scanner de secrets não encontra credenciais; testes confirmam mascaramento; o fluxo de desconexão inutiliza tokens locais; health checks não expõem segredos.

16.16 Passo 14 — Dockerizar e documentar a execução

Objetivo: permitir que outra pessoa ou agente execute o projeto de forma previsível.

Ações: criar Dockerfile; adicionar compose somente se necessário; montar volume para SQLite em desenvolvimento; documentar variáveis; adicionar comandos de build, teste, execução e migration; incluir diagrama atualizado.

Entregáveis: imagem reproduzível, instruções locais e configuração sem segredos.

Verificação: clone limpo consegue construir e iniciar o serviço; migrations executam; testes funcionam dentro do ambiente documentado.

16.17 Passo 15 — revisão de portfólio e CI/CD

Objetivo: preparar o projeto para apresentação profissional sem publicar dados reais.

Ações: revisar README, arquitetura, decisões, limitações, screenshots da documentação, exemplos curl, cobertura de testes e histórico de commits; configurar CI para build, testes, análise estática e secret scanning; adicionar release notes.

Entregáveis: repositório apresentável, pipeline de validação e documentação de contribuição.

Verificação: CI passa em pull request; o projeto explica claramente o que a API oficial permite e o que deliberadamente não faz.

16.18 Passo 16 — validar migração experimental para PostgreSQL

Objetivo: comprovar que a separação de persistência foi bem executada.

Ações: criar profile PostgreSQL; adicionar driver e configuração fora do profile SQLite; adaptar migrations se necessário; executar testes de persistência e casos de uso nos dois bancos; registrar diferenças de concorrência e índices.

Entregáveis: relatório de compatibilidade e profile experimental.

Verificação: domínio, serviços, portas e controllers permanecem iguais; alterações ficam limitadas à configuração, adaptador de persistência e migrations.

17. Checklist de passagem entre etapas

Uma etapa só deve ser marcada como concluída quando todos os itens a seguir forem verdadeiros:

- o objetivo está demonstrado por teste ou evidência reproduzível;
- o código compila e os testes relevantes passam;
- o comportamento está documentado;
- não existem secrets ou dados reais no repositório;
- erros e limites conhecidos estão registrados;
- a etapa não introduziu dependência indevida entre domínio, LinkedIn ou SQLite;
- as migrations foram testadas em banco limpo;
- o próximo agente consegue entender o estado atual apenas lendo o README, o planejamento e o relatório da etapa.

Cada agente deve deixar um registro curto em `docs/progress/STEP-XX.md` , contendo objetivo, alterações, comandos, testes, decisões, pendências e próximo passo. Esse registro cria uma memória operacional do projeto e evita que agentes posteriores repitam trabalho ou alterem decisões sem contexto.

Referências

- [1] LinkedIn User Agreement
- [2] Prohibited software and extensions — LinkedIn Help
- [3] Share on LinkedIn — Microsoft Learn
- [4] Getting Access to LinkedIn APIs — Microsoft Learn
- [5] Easy Apply limits — LinkedIn Help